ADR 001: Technology Stack for Mobile & AR Rice Disease/Pest Detection

Status: Approved
Date: 2026-02-02
Author: AI & CV Research Team
Scope: End-to-end stack for development, training, optimization and deployment of AI models for rice disease and pest detection on entry-level Android devices and augmented reality (AR) glasses.[1] [2]

1. Context and Problem

The project aims to detect diseases and pests in rice crops in real time, directly in the field, using cameras on low-cost smartphones and AR glasses, without relying on cloud connectivity.[1] [2]
Recent literature shows that lightweight convolutional and detection models (MobileNetV2/V3, ShuffleNetV2, efficient YOLO variants) can reach >90–95% accuracy on rice leaf diseases and pests while remaining deployable on mobile hardware with 2–4 GB RAM.[2] [3] [4] [5] [6] [7] [1]
The architecture must therefore support: (1) flexible research and training, (2) model compression and optimization, and (3) efficient offline inference on Android and AR devices.[8] [1] [2]

2. Decision

We adopt the following stack as the project standard:

1. **Languages**
   - Python 3.10+ for model development, training, experimentation and optimization.[4] [1]
   - Kotlin (with occasional Java interop) for Android apps on smartphones and Android-based AR glasses.[8]
   - C++ only where needed for performance-critical native components or TFLite custom ops.[8]
2. **ML / CV Frameworks and Libraries**
   - PyTorch 2.x as the **primary training framework** for CNN and YOLO-style models.[9] [1] [4]
   - Ultralytics YOLO for object detection experiments (YOLOv8-nano and similar lightweight variants).[7] [4]
   - torchvision, Albumentations and scikit-learn for data augmentation and evaluation.[1] [4]

3. **Model Export and Optimization**
    - ONNX as an intermediate exchange format from PyTorch.[8]
    - TensorFlow Lite (TFLite) as the **primary runtime format** for mobile and AR deployment.[8]
    - TFLite post-training quantization (int8/float16) and pruning to reduce model size and latency, following approaches used in lightweight rice disease models such as LiSA-MobileNetV2 and YOLO-derived architectures.[6] [4] [7] [1]
4. **Mobile and AR Deployment**
    - Android Studio + Kotlin for smartphone apps; integrate TFLite via the official Android API and hardware delegates (NNAPI/GPU).[8]
    - ARCore (or equivalent OpenXR-based SDK, depending on device) for AR tracking and scene understanding on Android-compatible AR glasses.[8]
    - Unity as 3D/AR engine where recommended by AR device vendors, integrating TFLite through available plugins or native interfaces.[8]
5. **Computer Vision Pre/Post-processing**
    - OpenCV (Python and Android) for image preprocessing, color normalization and geometric transforms.[1]
    - Optional: MediaPipe for high-level real-time pipelines if it reduces engineering effort for camera + ML integration.[8]

This stack is the reference for all new components unless an exception is explicitly documented in a separate ADR.[2] [1] [8]

3. Technical Criteria

The technologies were selected against the following **explicit criteria**:

1. **Computational Efficiency and Model Size**
    - Target architectures must be compatible with entry-level devices and AR glasses: typically <10M parameters and <1–2 GFLOPs where possible.[3] [5] [2] [4] [6] [7] [1]
    - Studies on rice disease/pest detection report that lightweight MobileNet- and ShuffleNet-based models, as well as small YOLO variants, achieve high accuracy (often >95%) while remaining efficient for mobile deployment.[5] [2] [4] [6] [7] [1]
2. **Offline Operation**
    - All inference must run entirely on-device, since rural deployment locations often lack stable connectivity and on-device processing simplifies privacy and latency constraints.[2] [1]
    - TFLite is explicitly designed for on-device inference on Android and embedded devices, with optimized kernels and support for hardware delegates.[8]
3. **Compatibility with Entry-Level Mobile and AR Hardware**

- Target devices: Android phones with 2–4 GB RAM and mid-range SoCs, plus Android-based AR glasses using similar chipsets.
- Lightweight rice disease and pest models in recent work (e.g., LiSA-MobileNetV2, YOLO-DP, SSD-YOLO) are evaluated on hardware of comparable capacity, showing feasible real-time or near real-time performance.[4] [6] [7] [2] [1]

4. **Flexibility for Research and Evolution**
   - PyTorch is widely used in computer vision research and in recent comparative studies for rice disease detection, including CNNs, transformers and non-neural baselines.[9] [4] [1]
   - ONNX provides a vendor-neutral path to multiple runtimes, reducing lock-in and simplifying future experimentation with alternative inference engines.[8]
5. **Ecosystem, Documentation and Team Skills**
   - Python + PyTorch + TFLite is one of the most documented pipelines for edge AI, with abundant examples and community support.[4] [1] [8]
   - Kotlin is the current Google-recommended language for Android development, with strong tooling and sample projects for TFLite and ARCore.[8]

4. Restrictions and Constraints

1. **Hardware Constraints**
   - Memory and compute limitations restrict us to lightweight models; heavy backbones such as ResNet-50/101, VGG-16 or full-sized YOLO variants are not acceptable for deployment, even if used as baselines in research.[3] [1] [4]
   - AR glasses introduce stricter power and thermal constraints, which limit sustained GPU usage and require aggressive optimization and frame-rate management.[8]
2. **Connectivity Constraints**
   - Cloud inference (e.g., TensorFlow Serving, remote GPU clusters) is excluded from the operational design; connectivity, when available, will be reserved for updates, not real-time inference.[2] [1]
3. **Platform Constraints**
   - The initial release targets Android phones and Android-compatible AR glasses; iOS and Apple-specific AR platforms (Core ML, ARKit, Vision Pro) are out of scope for this phase.[8]
   - Some AR devices expose limited APIs; in those cases, Unity or the vendor SDK will constrain how TFLite can be integrated.[8]
4. **Data and Privacy Constraints**
   - Images may inadvertently capture people or sensitive locations; on-device processing supports privacy by design and simplifies compliance.

5. Options Considered and Justification

5.1 Training Framework

Option A – PyTorch (Selected)

- Pros:
  - Strong presence in recent rice disease detection research and in general CV, enabling quick reproduction of literature and experimentation with novel architectures.[9] [1] [4]
  - Rich ecosystem for detection (Ultralytics YOLO) and augmentations suited to agricultural imagery (e.g., illumination and background variation).[4] [9]
  - Straightforward export to ONNX, which can then be converted to TFLite.[8]
- Cons:
  - Quantization and deployment require an additional conversion step compared to a pure TensorFlow pipeline.[8]
- **Decision:** Chosen as the primary research and training framework.

Option B – TensorFlow/Keras (Not Selected as Primary)

- Pros:
  - Tight integration with TFLite and many official mobile examples.[8]
- Cons:
  - Less aligned with some of the latest YOLOv8-based rice detection work, which is primarily PyTorch-based.[9] [4]
- **Decision:** May be used for specific tasks or experiments but is not the default.

5.2 Inference Runtime

Option A – TensorFlow Lite (Selected)

- Pros:
  - Optimized for on-device inference with support for CPU, GPU and NNAPI delegates on Android.[8]
  - Widely used for mobile agricultural and plant disease detection applications, matching project use cases.[1] [2] [4]
  - Supports post-training quantization techniques used successfully in lightweight rice disease models, improving speed and reducing model size.[6] [7] [4]
- Cons:
  - Conversion pipeline (PyTorch → ONNX → TFLite) adds complexity.[8]
- **Decision:** Selected as the main runtime for phones and AR devices.

Option B – ONNX Runtime (Not Selected as Primary)

- Pros:
  - Cross-platform and can run on many environments.[8]
- Cons:
  - On Android, hardware acceleration and ecosystem maturity are generally better for TFLite, especially on low-end devices.[8]
- **Decision:** Retained as a secondary option for special cases but not standard.

5.3 Mobile Application Technology

Option A – Native Android (Kotlin + TFLite) (Selected)

- Pros:
  - Best access to device capabilities (camera, storage, NNAPI, ARCore) and to official TFLite examples/documentation.[8]
- Cons:
  - Requires Android-specific skills; not cross-platform for iOS.
- **Decision:** Selected as the default, given the project's Android focus.

Option B – Cross-Platform UI (Flutter/React Native) (Not Selected as Primary)

- Pros:
  - Single codebase for multiple platforms.
- Cons:
  - More complex ML integration and weaker direct access to some low-level APIs and accelerators.[8]
- **Decision:** Not chosen for the initial phase; may be reconsidered if multi-platform UI needs become critical.

5.4 AR Technology

Option A – ARCore + Unity + TFLite (Selected)

- Pros:
  - ARCore offers robust tracking and plane detection on Android and is commonly supported by AR devices.[8]
  - Unity is widely used in AR projects and often recommended by device vendors; TFLite integrations and examples are available.[8]
- Cons:
  - Added complexity and possible performance overhead compared with pure Android apps.

- **Decision:** Selected as the primary AR combination, with adjustments according to specific AR hardware SDKs.

6. Consequences

Positive:

- A clear, repeatable pipeline (PyTorch → ONNX → TFLite → Android/AR) that aligns with how recent rice disease and pest detection models are implemented and deployed in the literature.[7] [6] [2] [1] [4] [8]
- Strong support for offline, low-latency inference on entry-level devices using TFLite and lightweight architectures (MobileNet, ShuffleNet, small YOLO variants).[5] [6] [7] [2] [1] [4]
- Reduced privacy and connectivity risks, since all real-time inference remains on-device.

Negative / Risks:

- Android-centric and TFLite-centric choices may require additional work to support iOS or non-Android AR platforms in the future (e.g., Core ML conversions).[8]
- Accuracy can drop slightly after quantization, which must be monitored against benchmarks from rice disease detection studies.[6] [7] [4]
- AR device diversity (different vendors and SDKs) can complicate integration and performance tuning.[8]

7. Key Sources Used

- Surveys and reviews on crop and rice disease/pest detection:[10] [11] [12] [13] [1]
- Rice-specific lightweight and mobile-oriented models (MobileNet, ShuffleNet, YOLO variants):[3] [5] [7] [2] [6] [1] [4]
- General guidance on architecture decision records and mobile/edge AI architecture:[14] [15] [16] [8]
- Project context: focus on efficient, offline detection on entry-level mobile devices for rice crops.

⁂

1. interests.agriculture_tech.rice_disease_detection
2. https://pmc.ncbi.nlm.nih.gov/articles/PMC11253532/
3. https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2025.1643096/full
4. https://ar5iv.labs.arxiv.org/html/1812.01043
5. https://www.arxiv.org/pdf/2408.01752.pdf
6. https://www.techscience.com/cmc/v75n2/52135
7. https://pmc.ncbi.nlm.nih.gov/articles/PMC12370686/
8. https://pmc.ncbi.nlm.nih.gov/articles/PMC12528418/

9. https://docs.cloud.google.com/architecture/architecture-decision-records
10. https://arxiv.org/html/2501.06740v1
11. https://pmc.ncbi.nlm.nih.gov/articles/PMC10088765/
12. https://arccjournals.com/journal/indian-journal-of-agricultural-research/A-6061
13. https://www.rcs.cic.ipn.mx/2023_152_10/Pests Detection in Agricultural Crops Using Computer Vision.pdf
14. https://repositorio.iscte-iul.pt/bitstream/10071/26147/1/article_90301.pdf
15. https://github.com/joelparkerhenderson/architecture-decision-record
16. https://adr.github.io
17. https://adr.github.io/adr-templates/